# Introduction to Parsing Theory

*Compilers need to parse. A parser is just code, so to build one, we could just wing it. But a theory helps us understand parsing at a deep level, enabling us to write better, faster, and provable correct parsers. Let's learn!*

CONTENTS

What is Parsing Again? • Lexical Analysis • Approaches to Parsing • Recursive Descent • Table-Based Parsing • Parsing Expression Grammars • Real-World Issues • Resources • Summary

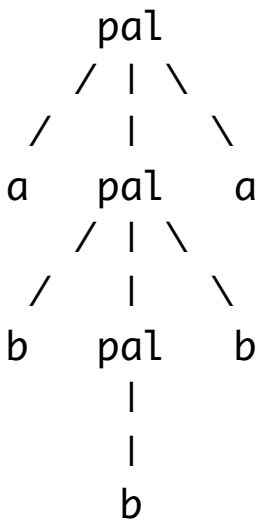
## What is Parsing Again?

**Parsing** is the process of uncovering the derivation, or derivation tree, of a string from a grammar.

For example, given the grammar:

| *pal* ⟶ ("a" \| "b" \| "a" *pal* "a" \| "b" *pal* "b")? |
|---|

we **parse** the string ***abbba*** into the derivation tree:




In a real programming language, we generally don't make a derivation tree for each character in the source program. Instead, we first group characters into **tokens** and we consider the tokens as the symbols.

TODO

The pre-processing of the source program via tokenization is so common and so natural and so important that there is a kind of separate (simpler) theory and practice that deals with this, called **lexical analysis.** Let's briefly talk about that first before getting into the richer theory of parsing.

# Lexical Analysis

Tokenization, or lexical analysis, is the process of forming tokens out of the source code character stream. You basically look at the character stream character-by-character, skipping whitespace and comments, and collecting characters into tokens. Sometimes you might need to "look ahead" a little, or "back up" as you go through the character stream.

Generally, each token will have a type, a lexeme, and line and column position. Types of tokens include:

- keyword
- identifier
- numeric literal
- string literal
- punctuation

The **lexical grammar** is the part of the language specification that shows how tokens are formed from characters. Negative lookahead is common! An example might be:

TODO

Sometimes people find it helpful to visualize how a **lexer** works via a Finite Automata:

TODO

Writing a lexer by hand is possible. For the simple lexical grammar shown above, a hand-crafted lexer might be:

TODO

In general, lexers are concerned with issues such as:

- Case sensitivity (or insensitivity)
- Whether or not blanks are significant
- Whether or not newlines are significant
- Whether comments can be nested

Errors that might occur during scanning, called **lexical errors** include:

- Encountering characters that are not in the language's alphabet
- Too many characters in a word or line (yes, such languages do exist!)
- An unclosed character or string literal
- An end of file within a comment

Once you have a lexer written and deployed, you can now work entirely with token streams. Parsing now works entirely at the level of the phrase grammar. The parser's job is to uncover the derivation tree, or parse tree, whose leaves are now tokens, not characters.

# Approaches to Parsing

A language's syntax is normally defined with a CFG (or the equally powerful BNF, EBNF, Syntax Diagram), or a PEG. A compiler uses the grammar to **parse** the token sequence, that is, *to determine if the token string can be generated by the grammar, and produce the syntax tree for the token sequence*. People have been studying parsing for decades! Here are some things they figured out:

- Context-sensitivity is too hard. Separate it out. Just focus on the context free parts of the language in the parser.
- There's a desire for grammars for real programming languages to be **unambiguous**.
- Parsers should be fast! There is a theorem that says every CFG can be parsed in $\Theta(n^3)$ time. That is too slow. Perhaps we should look for restricted CFGs that can be parsed more efficiently.
- There is! People have figured out and defined the classes of $LL$ grammars and $LR$ grammars! These are restricted CFGs that can be parsed in $\Theta(n)$ time. Slay!
- Massive amounts of theory have been generated around parsing techniques for $LL$ and $LR$ grammars, since the mid-20th century. $LL$ mechanisms are easy but the class of $LL$ languages is much smaller than $LR$. But $LR$ methods are really tough to follow and not meant for human understanding.
- PEGs hit the scene in 2004 and are awesome! They are a little different from CFGs. They can be parsed in linear time with a packrat parser, but at the cost of a lot of additional space. Ohm adds some power to PEGs (left recursion) so you *might* encounter super-linear parse times.

That's some good background info to start a study of parsing. But wait, first, what do $LL$ and $LR$ mean?

**$LL(k)$ Grammar**

A grammar that can be used to drive a leftmost derivation by reading the input left-to-right, examining only the next $k$ symbols, and never backing up.

**$LR(k)$ Grammar**

A grammar that can be used to drive a rightmost derivation by reading the input left-to-right, examining only the next $k$ symbols, and never backing up.

Um, are there more practical descriptions of these language classes? Pretty much, yeah. Try these:

**$LL(k)$**

Locate all the "choice points" in the grammar (these are easy to find in the equivalent syntax diagram). Determine the maximum number of tokens, $k$, required to know exactly which path to take over all choice points. If $k$ is finite, the grammar is LL(k).

**$LR(k)$**

The set of $LR$ languages is exactly equivalent to the set of languages recognizable by a DPDA. $LR$ languages are exactly the DCFLs. An $LR(k)$ grammar is a CFG for a language that is a DCFL.

An $LL$ and $LR$ Fact Sheet:

| LL Grammars | LR Grammars |
|---|---|
| • Subset of CFGs<br>• Never ambiguous<br>• Parsable in Linear Time<br>• Left-to-right, Leftmost derivation<br>• Top-down<br>• Predictive<br>• Expand-Match | • Subset of CFGs<br>• Never ambiguous<br>• Parsable in Linear Time<br>• Left-to-right, Rightmost derivation<br>• Bottom-up<br>• Not Predictive<br>• Shift-Reduce |

| | |
|---|---|
| • Parsers can be hand-made<br>• Can give nice error messages<br>• Can not handle left recursion<br>• Less "powerful" than LR | • Nearly impossible to write parsers by hand<br>• Must work hard to give good errors<br>• Left recursion is desirable!<br>• More "powerful" than LL |

Wow! That was a lot. Details are coming. Read on.

> ***Exercise**: Why is it that a grammar with left-recursion can not be LL(k) for any k?*

> ***Exercise**: For more on **LL** and **LR** see this BNF article with a great little section on parsing.*

We've made syntax diagrams to describe all the legal programs in the language, but how to we follow the diagrams to "match" or "parse" a candidate program? That is, given a program (a candidate string), how, exactly, do we figure out how it is matched by, or generated by, the syntax diagrams? How do we **parse**?

There have been hundreds of books and thousands of papers on the topic of parsing over the years. Here we just need to highlight a few things.

# Hand-Crafted Parsers

Because a parser is just a program that turns source code into a parse tree, you can just write on by hand. The most common kind of hand-crafted parser uses the technique of recursive descent, which you can read about at Wikipedia. You kind of walk through each of the syntax diagrams.

CLASSWORK

> I've written a recursive descent parser as part of an Astro compiler. We'll take a very brief look at it in class.

# Parser Generators

A recursive descent parser generally requires a highly restricted syntax, one that is line with a class of languages known as the $LL(k)$ languages. These languages are a proper subset of the $LR(k)$ languages (also known as the deterministic context-free languages), which are used in popular parser generators such as YACC, Bison, and ANTLR, among others. While these systems are quite popular and famous, we won't be covering them here. Do read about them, though.

We will see $LL(k)$ and $LR(k)$ later in the course, though.

# Analytic Grammars

Another approach is to use a specialized kind of grammar designed specifically for parsing. So instead of using traditional **generative grammars**, covered previously in our notes on Language Theory (which specify a language by giving a set of rules to **generate** all and only those strings in a language), we can use an **analytic grammar**, in which the rules specify how to **match** a candidate string.

The biggest difference between generative and analytic grammars is in the handling of alternatives. In a generative grammar $G$, $L(G)$ is contains all strings that *can* be generated by applying rules whenever possible. For example, the rule $S \rightarrow A \mid B$ means you can replace an $S$ with an $A$ or a $B$, so you can generate both. So the grammar:

```
s ⟶ "a" | "ab"
```

describes the language $\{a, ab\}$.

In an analytic grammar, the rules are deterministic; you don't get to pick one or the other. So $A \ / \ B$ means: "Try to match $A$. If you can, the rule matches and you are done. Otherwise (and only otherwise), try to match $B$". So:

```
s ⟵ "a" / "ab"
```

recognizes the language $\{a\}$, but:

```
s ⟵ "ab" / "a"
```

recognizes the language $\{a, ab\}$.

Note the tradeoff here. Determinism means there's never ambiguity (a string can only parse one way, if it can at all), which is great! But the ordering of the rules matters, so you have to be careful to do what you want. Here's another illustration. If we wanted an analytic grammar for Astro, the rule:

```
Primary ⟵ num / id / Call / ...
```

would be wrong, as it will *never* match a call!

> ***Exercise**: Why, exactly, must the Call alternative precede the id alternative?*

CLASSWORK

> This might be deep, so we'll try a few more examples.

Unlike generative grammars, analytic grammars feature special **lookahead operators**, including the **negative lookahead operator** (~) and the **positive lookahead operator** (&). This enables the definition of non-context-free languages, despite having only one symbol on the left-hand side! Here's an analytic grammar for the non-context-free language $\{a^n b^n c^n \mid n \geq 0\}$:

```
s ⟵ & (x "c") "a"+ y
x ⟵ "a" x? "b"
y ⟵ "b" y? "c"
```

# Parsing Expression Grammars (PEGs)

There are various formalisms of analytic grammars (Wikipedia identifies several); the most popular is the Parsing Expression Grammar (PEG). To read a PEG description, pronounce the operators as follows:

- `A B` : match A then match B
- `A / B` : match A. If it matched, success. Otherwise, match B
- `A?` : match A zero or one times
- `A*` : match A zero or more times
- `A+` : match A one or more times
- `&A` : match A without consuming it (but it must match)
- `~A` : match A without consuming it. If A matches then ~A fails. If A fails ot match, then ~A succeeds
- `end` : matches the end of the string
- Quoted text: successively match each symbol exactly

Note how PEGs use `/` instead of `|`, perhaps to emphasize the deterministic nature of matching.

Strictly speaking, it should be impossible in a PEG to have **left-recursive rules** like:

$$Exp \longleftarrow Exp \text{ "+" } Term \ / \ Term$$

given that "in order to match an Exp, you much first try to match an Exp." It's sounds circular. In fact it is, and traditionally, PEG implementations suggested using the rule:

$$Exp \longleftarrow Term \ (\text{"+" } Term)^*$$

instead. However, it turns out a proper parsing technique (using a **packrat parser**) can deal with it! It's not easy, but you can read the original paper by Alex Warth et al or this nice and detailed article by Guido van Rossum to see how it's done.

***Check your understanding***

*Analytic grammars:*

- ✅ *Are designed for parsing (recognition) rather than generation of languages.*
- ✅ *Are written with only a single variable on the left hand side of each rule.*
- ✅ *Use ⟵ rather than ⟶ to define rules.*
- ✅ *Execute alternatives left-to-right and once you get a match, you keep it, even if it leads to a dead end.*
- ✅ *Have powerful lookahead operators, thanks to deterministic execution.*
- ✅ *Can handle left-recursive rules.*

Try this exercise before moving on:

***Exercise**: Show that the following generative grammar for assignment statements and if-then-else statements in a programming language:*

```
S → "a" | "i" "x" "t" S ("e" S)?
```

*is ambiguous, but that the ambiguity can be resolved by rewriting as a PEG:*

```
S ← "a" / "i" "x" "t" S "e" S / "i" "x" "t" S
```

*How, exactly, is the ambiguity resolved? (Historical note: this is the dangling else problem, a classic in parsing theory.)*

Now let's put everything together to come up with a Parsing Expression Grammar for Astro, that (1) properly separates lexical and phrase rules, (2) deals properly with precedence and associativity, (3) embraces left-recursion, and (4) correctly handles reserved words.

```
Program     ←  Statement+
Statement   ←  Assignment / PrintStmt
Assignment  ←  id "=" Exp ";"
PrintStmt   ←  print Exp ";"
Exp         ←  Exp ("+" | "-") Term / Term
Term        ←  Term ("*" | "/" | "%") Factor / Factor
Factor      ←  Primary ("**" Factor)? / "-" Primary
Primary     ←  num / Call / id / "(" Exp ")"
Call        ←  id "(" (Exp ("," Exp)*)? ")"
num         ←  digit+ ("." digit+)? (("E"|"e") ("+"|"-")? digit+)?
print       ←  "print" ¬idchar
id          ←  ¬print letter idchar*
idchar      ←  letter | digit | "_"
space       ←  " " | "\t" | "\r" | "\n" | "//" (¬"\n" any)*
```

> ***Lexical and Phrase Variables***
>
> *Ohm also adopts the convention of starting the names of lexical variables with a lowercase letter and phrase variables with an uppercase letter.*

# Recursive Descent

# Table-Based Parsing

# Parsing Expression Grammars

PEGs are designed for direct parsing: there's no need to "extract" any kind of machine. The matching process is built into the grammar rules.

TODO

Must reads:

- Wikipedia's article
- The original paper by Bryan Ford
- Short blog post on CFGs vs PEGs
- Using PEGs Derivatives (Paper that pointed out a huge bug in Ford's original paper)
- 2014 paper relating CFGs and PEGs
- 2020 paper on The computational power of PEGs

# Real-World Issues

The language designer will have to take into account any ambiguities in the language description. For example, in C, the expression `(x)–y` hase two different syntactic interpretations: if `x` refers to a type, then its a cast of a negation, otherwise it is a subtraction. You can't tell at this phase.

Errors that occur during parsing, called **syntax errors** arise from malformed token sequences such as:

- `j = 4 * (6 – x;`
- `i = /5`

- `42 = x * 3`

TODO: non-syntax errors

# Resources

# Summary

We've covered:

- ✅ What parsing is
- ✅ Lexical analysis vs. syntax analysis
- ✅ Various approaches to parsing
- ✅ The method of recursive descent
- ✅ Parsing based on Context-Free Grammars
- ✅ A good deal about Parsing Expression Grammars
- ✅ Where to learn more